Ferric FX: a web-based audio effects processor in p5.js

David Cardozo

5 July 2026

The application

(499 words)

Ferric FX is a web audio application built with p5.js and its p5.sound library. A pre-recorded sound file plays through a fixed processing chain (low-pass filter, waveshaper distortion, dynamic compressor, reverb and master volume); the result is sent to the output device, recorded to a WAV file, or saved into a sounds library.

The chain is built once at start-up in `audioEngine.js`. Every p5.sound effect auto-connects to the speakers, so each node is first `disconnect()`ed and then wired in series (`source → p5.LowPass → p5.Distortion → p5.Compressor → p5.Reverb → p5.Gain → output`), the fixed signal flow shown in Figure 2. Two `p5.FFT` analysers tap the chain's head and tail to draw the *Spectrum In* and *Spectrum Out* displays, so every edit is immediately visible. Transport buttons (play/pause, stop, loop, skip to start) drive the current `p5.SoundFile`.



Figure 1: The interface: transport and REC, signal-flow strip, live in/out spectra, per-stage editor, sounds library.



Figure 2: Internal signal flow. A single fixed chain carries the source through the five effect stages to the output device; `audioEngine.js` builds it once and thereafter only pushes parameter changes into the live nodes.

The effects

Low-pass filter (`p5.LowPass`, via `freq()` and `res()`) attenuates everything above its cutoff at 24 dB/oct, while resonance emphasises the corner frequency. Figure 3 shows a broadband input reduced to its lowest bands by a 400 Hz cutoff.



Figure 3: Low-pass at 400 Hz, fully wet, other stages bypassed: input energy above the cutoff vanishes from the output.

Waveshaper distortion (`p5.Distortion`, `set(amount, oversample)`) pushes the signal through a nonlinear transfer curve. Unlike a filter it creates new spectral content: harmonics of the input partials. In Figure 4 the same filtered signal is driven hard and energy reappears *above* the cutoff, generated entirely by the distortion. Oversampling (2×/4×) tames the aliasing.

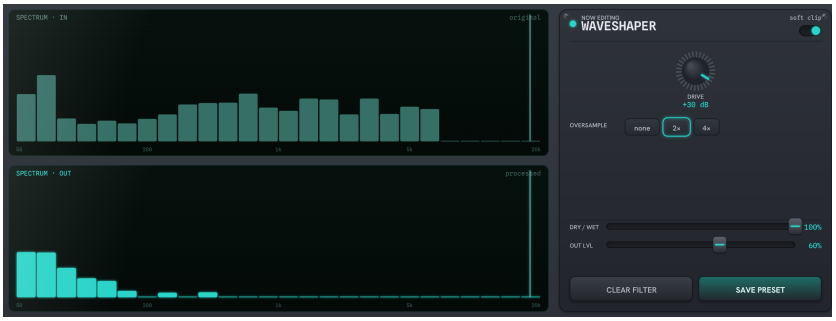


Figure 4: The waveshaper re-enabled after the same 400 Hz filter: harmonics reappear above the cutoff.

Dynamic compressor (`p5.Compressor`: threshold, ratio, knee, attack, release) reduces dynamic range, adding no new frequencies; magnitudes stay more constant over time, raising average loudness.

Reverb (`p5.Reverb`, `set(duration, decay, reverse)`) convolves the signal with a decaying impulse response, smearing energy in time and filling the gaps between transients: a denser, sustained spectrum.

Master volume (`p5.Gain`) scales the mix uniformly, so the spectrum keeps its shape at a different level. Each stage also has DRY/WET and OUT LVL faders; bypass is implemented as `drywet(0); amp(1)`, passing the dry signal at unity so the graph never needs rewiring.

Two enhanced filter options

A *feedback delay* (`p5.Delay`) would add echoes; at short delay times the delayed copy interferes with the direct signal, carving comb-filter notches spaced at 1/delay Hz across the spectrum.[1] A *multi-band equaliser* (`p5.EQ`) would allow surgical boosts and cuts at chosen frequencies rather than the low-pass's single downward slope.[2] Both are p5.sound nodes that could be patched into the chain unchanged.

[1] `p5.Delay` reference: https://p5js.org/reference/p5.sound/p5.Delay. On the comb filtering produced by short feedback delays, see J. O. Smith III, *Physical Audio Signal Processing*, W3K Publishing, 2010, §"Feedback Comb Filters".

[2] `p5.EQ` reference: https://p5js.org/reference/p5.sound/p5.EQ. Equaliser and filter effect design is treated in J. D. Reiss and A. P. McPherson, *Audio Effects: Theory, Implementation and Application*, CRC Press, 2015.

The sounds library

Six pre-recorded WAVs load into the library panel at start-up. Selecting a card loads it into the chain; EDIT opens its stored effect configuration in the editor, CLR resets it to a dry source, DEL removes it. REC records the processed master bus with `p5.SoundRecorder`; stopping opens the capture dialog (Figure 5) where the take can be auditioned, renamed, exported as a WAV via `save()`, or added to the library with its effect settings baked in.

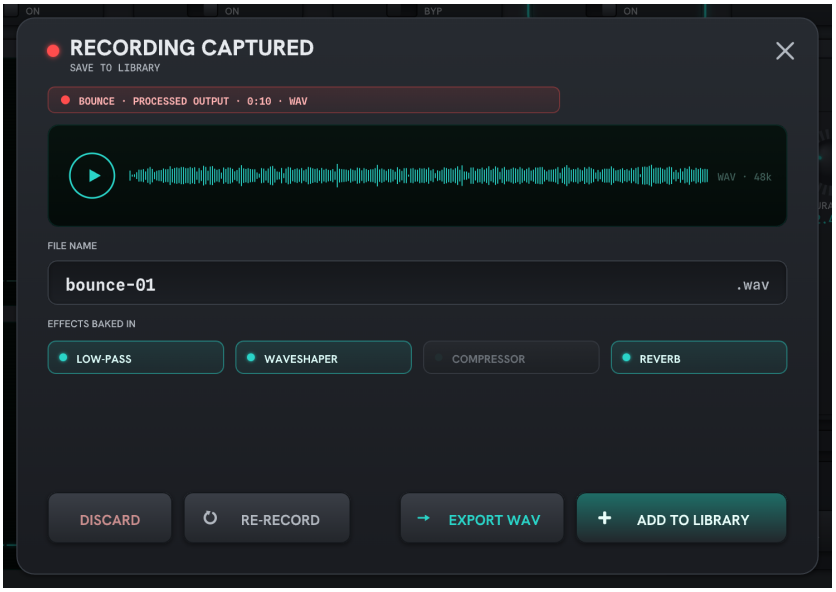


Figure 5: The capture dialog: waveform preview, file name, the effects baked into the take, export/save actions.

The new card lists its stored effects (`LPF · WSH · RVB`, Figure 6). SAVE PRESET persists the live settings; the library index lives in `localStorage` and recorded or imported audio in IndexedDB, so sounds *and* their effect configurations survive a reload to be re-edited, cleared or deleted later.



Figure 6: The library after saving: `bounce-01` carries its baked-in effect chain.

Appendix: constructing the chain

How `audioEngine.js` builds the graph of Figure 2 once at start-up (abridged):

```
nodes = {
  lowpass:    new p5.LowPass(),
  waveshaper: new p5.Distortion(),
  compressor: new p5.Compressor(),
  reverb:     new p5.Reverb(),
  masterGain: new p5.Gain(),
};
// undo each effect's default route
// to the speakers
[nodes.lowpass, nodes.waveshaper,
 nodes.compressor, nodes.reverb]
  .forEach((n) => n.disconnect());
// wire the fixed series chain
nodes.lowpass.connect(nodes.waveshaper.input);
nodes.waveshaper.connect(nodes.compressor.input);
nodes.compressor.connect(nodes.reverb.input);
nodes.reverb.connect(nodes.masterGain.input);
nodes.masterGain.connect(); // -> output device

fftIn = new p5.FFT();  // Spectrum In: the source
fftOut = new p5.FFT(); // Spectrum Out: post-FX
fftOut.setInput(nodes.masterGain);
recorder = new p5.SoundRecorder();
recorder.setInput(nodes.masterGain); // REC taps here
```